RAG & AI Architecture Reference Manual

System Design, Data Ingestion, and Retrieval Fundamentals

> **Document Overview:** This document serves as a test standard for document loaders, text splitters, and vector store indexing engines built with LangChain and Python.

1. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an architectural framework designed to enhance the output of Large Language Models (LLMs) by integrating external knowledge sources. Standard LLMs rely solely on their pre-trained parameters, which can lead to hallucinated facts or outdated answers. By introducing a retrieval component, RAG enables models to ground their answers in authoritative, up-to-date data.

2. Core Components of a RAG Pipeline

- **Document Loading & Parsing:** Converting raw unstructured text, PDFs, or web pages into standardized document objects.
- **Text Chunking:** Splitting large documents into smaller, semantically coherent segments to fit within model context windows.
- **Vector Embeddings:** Mapping text chunks into continuous high-dimensional vector spaces using specialized embedding models.
- **Vector Store Retrieval:** Querying indexed vectors using cosine similarity or HNSW index algorithms to fetch top-k relevant contexts.
- **Contextual Generation:** Passing retrieved chunks alongside user queries into the LLM to synthesize precise responses.

3. Fundamentals of Neural Networks

Artificial Neural Networks (ANNs) are computing systems inspired by the biological neural networks that constitute animal brains. An ANN is based on a collection of connected units or nodes called artificial neurons, which loosely model the neurons in a biological brain. Each connection can transmit a signal to other neurons across multiple layers.

4. RAG Component Specifications

Component	Primary Function	Recommended Tool / Package
Document Loader	Parses raw text, markdown, and PDF files into Document objects	`langchain_community.document_loaders`
Text Splitter	Recursively splits text using character and semantic boundaries	`RecursiveCharacterTextSplitter`
Embeddings	Converts text chunks into numerical vector representations	`HuggingFaceEmbeddings` / OpenAI
Vector Store	Indexes and retrieves high-dimensional vector embeddings	Chroma / FAISS / Qdrant